LAB TASKS: INTELLIGENT AGENTS & ENVIRONMENT MODELING

LAB OBJECTIVE

1. To Create AI Workflow using Node automation tool
2. To implement and analyze fundamental intelligent agent architectures and environment models using Python, emphasizing computational reasoning, agent–environment interaction, and decision-making behavior.

PART A: N8N

TASK 1: INTELLIGENT EMAIL WORKFLOW | AI-DRIVEN INBOX ORCHESTRATION AGENT (USING N8N)

Case Study

Background

XYZ is a faculty member in the Department of Artificial Intelligence & Data Science. On a daily basis, she receives a high volume of heterogeneous email traffic, including:

- Lab-related emails from students that frequently contain missing information, unclear subject lines, or incomplete explanations
- Departmental and administrative emails concerning meetings, scheduling, and time-sensitive academic matters
- Miscellaneous emails that do not contribute to her instructional or academic responsibilities

Manual inspection, prioritization, and response formulation for such emails introduces significant cognitive overhead and adversely impacts instructional productivity. *To mitigate this inefficiency, an **automated, AI-assisted email management agent** is required.*

Objective

Design and implement an **AI-powered email orchestration workflow** using **n8n** that autonomously:

- Performs **semantic intent classification** of incoming emails
- Dynamically routes emails based on inferred intent
- Generates contextualized draft responses only when necessary
- Applies priority-based labels without unnecessary reply generation

Functional Requirements

The system must classify all incoming emails into **exactly one** of the following semantic categories:

- **Lab_Query**
- **Departmental_Meeting (High Priority)**
- **Ignored**

Technology Stack

- **Gmail Trigger Node:** Event-driven detection of new inbound emails
- **Set Node:** Input sanitization and feature reduction
- **OpenAI (GPT):** Natural Language Understanding (NLU) for intent classification
- **IF Node**: Conditional branching and control flow logic
- **OpenAI (GPT)**: Automated professional response synthesis
- **Gmail Draft Node**: Draft persistence without transmission
- **Gmail Label Node**: Priority-based inbox annotation

Workflow Specification

1. Event Trigger

Initiate workflow execution upon receipt of a new email in the Gmail inbox.

2. Input Preprocessing (Set Node)

Extract and retain only semantically relevant fields for downstream NLP processing:

- Email body
- Sender address
- Subject line

All extraneous metadata must be discarded to minimize prompt noise.

3. AI-Based Intent Classification (OpenAI Node: select "Message a Model")

Apply a large language model to infer the semantic intent of the email. Choose ***GPT-4O-MINI*** model.

System Prompt (STRICT: USE AS IS)

```
You are an email classifier for an academic instructor named XYZ.

Classify the email into ONLY one of the following labels:

1. Lab_Query

2. Departmental_Meeting
```

3. Ignored

Rules:

- Lab_Query: student asking about lab work, submissions, issues, marks, deadlines

- Departmental_Meeting: emails about meetings, schedules, departmental matters, urgent discussions

- Ignored: anything else

Return ONLY one word exactly from:

Lab_Query

Departmental_Meeting

Ignored

4. Conditional Routing Logic (IF Node)

Branch A: Lab_Query (Grab text: *{{ $json["output"][0]["content"][0]["text"] }}*) → Draft

5. Response Generation Node (OpenAI Node: select "Message a Model")

Invoke an OpenAI node to generate a **professionally worded draft email** requesting missing academic identifiers and issue clarification.

System Prompt

You are an assistant of XYZ, Instructor.

Write a professional draft email asking the student to provide:

- Registration Number

- Section

- Course / Subject Code

- Clear description of the lab issue

Tone: polite, professional, clear.

persist the output as a Gmail draft only (no auto-send).

Add user Role:

Subject: {{$json["Subject"]}}

Body:

{{$json["body"]}}

6. Create a Draft Node

Create a draft from Generated Response: {{$node["Message a model1"].json["output"][0]["content"][0]["text"].split('\n\n').slice(1).join('\n\n')}}

Set a subject: Lab Query - Missing Details

Set thread ID: {{$node["Gmail Trigger"].json["threadId"]}}

Set to email: {{$node["Gmail Trigger"].json["From"]}}

Branch B: Departmental_Meeting → Priority Label Assignment

Add a Label Node:

Set Message ID: {{$node["Gmail Trigger"].json["id"]}}

Label: High Priority

If the email pertains to departmental coordination or urgent academic matters:

- **Do NOT generate a reply**
- Apply Gmail label: **High Priority**

High Priority Definition

An email qualifies as high priority if it concerns departmental meetings, academic scheduling, urgent discussions, or requires immediate instructor attention.

Branch C: Ignored → Null Action

For all other emails:

- No draft generation
- No label assignment
- No further processing

Expected Draft Output (Lab_Query Only)

The workflow **must generate the following draft verbatim**:

Subject: Lab Query - Missing Details

Email Body:

Dear Student,

Thank you for reaching out regarding your lab query.

To assist you properly, please reply with the following details:

• Registration Number

• Section

• Course / Subject Code

• A clear description of your lab-related issue

Once these details are provided, I will be able to review your concern and guide you accordingly.

Regards,

XYZ

Instructor

Department of AI & Data Science

PART B: MODELING INTELLIGENT AGENTS & ENVIRONMENT USING PYTHON

TASK 2: MODEL-BASED AGENT WITH INTERNAL STATE TRACKING

Objective

Develop a **model-based agent** that maintains an internal representation of the environment to improve decision-making.

Problem Description

A smart vacuum agent operates in a two-room environment (Room A and Room B). Each room can be either **Clean** or **Dirty**.

Environment Characteristics

- Partially observable
- Deterministic

Procedure

1. Represent the internal state of each room.
2. Allow the agent to:
    - Update its internal model based on percepts
    - Decide whether to clean or move
3. Simulate multiple time steps.
4. Display changes in the internal state after each action.

Expected Learning Outcome

Students will learn how **internal memory and world models** enhance agent intelligence beyond reflex-based behavior.

TASK 3: UTILITY-BASED AGENT FOR DECISION MAKING

Objective

Design and implement a utility-based agent that selects an action by computing and maximizing a predefined utility function.

Problem Description

A delivery agent must choose **one route** from a **fixed set of available routes** to reach a destination. Each route is characterized by predefined numerical attributes. The agent evaluates each option and selects the route with the highest utility score.

Available Actions (Routes)

The agent can choose **only one** of the following actions:

- **Route A**
- **Route B**
- **Route C**

No additional routes or actions are allowed.

Route Attributes

Each route has the following predefined attributes:

Route	Distance (km)	Time (minutes)	Risk Level
Route A	10	15	Low
Route B	8	20	Medium
Route C	12	10	High

Environment Characteristics

- Fully observable
- Deterministic
- Static

Agent Characteristics

- Utility-based
- Preference-driven decision making

Procedure

1. Represent the available routes and their attributes in Python.
2. Convert/Assign qualitative risk levels (**Low, Medium, High**) into numerical values.
3. Define a utility function that combines:
    - Distance
    - Time
    - Risk
4. Compute a utility score for each route.
5. Select the route with the **maximum utility value**.
6. Display the utility scores and the selected route.

Constraints

- The agent must choose from **Route A, Route B, or Route C only**.
- The utility function must return a **single numerical value** per route.
- Hard-coded route selection is not allowed.

Expected Learning Outcome

Students will demonstrate the ability to model preferences and make rational decisions using a utility-based agent.

UNGRADED HOME TASKS:

TASK 4: PROBLEM-SOLVING AGENT IN A STATE-SPACE ENVIRONMENT

Objective

Design a problem-solving agent that navigates a discrete state-space environment to reach a predefined goal state using rule-based decision logic.

Problem Description

A robot is placed in a **linear grid environment** consisting of numbered positions from 0 to N. The agent starts at an initial position and must reach a specified goal position by selecting valid actions.

Environment Characteristics

- Fully observable
- Deterministic
- Static
- Discrete

Agent Capabilities

- Move Left
- Move Right

Procedure

1. Represent the environment as a list of states.
2. Define an initial state and a goal state.
3. Implement a problem-solving agent that:
    - Perceives its current state
    - Selects actions based on its relative distance from the goal
    - Terminates execution upon reaching the goal
4. Display the sequence of actions taken by the agent.

Expected Learning Outcome

Students will understand **state representation**, **goal formulation**, and **action selection** in classical problem-solving agents.

TASK 5: DETERMINISTIC ENVIRONMENT SIMULATION

Objective

Analyze agent behavior in a deterministic environment where actions always produce predictable outcomes.

Problem Description

A delivery agent moves on a grid to deliver a package. Each action (up, down, left, right) always results in the same state transition.

Procedure

1. Define a grid-based deterministic environment.
2. Implement an agent that navigates from start to destination.

3. Log state transitions after each action.
4. Verify that repeated runs yield identical results.

Expected Learning Outcome

Students will understand the defining characteristics of **deterministic environments** and their impact on agent planning.

TASK 6: STOCHASTIC ENVIRONMENT MODELING

Objective

Simulate a **stochastic environment** where action outcomes are probabilistic rather than guaranteed.

Problem Description

A navigation agent attempts to move in a grid. However, due to environmental uncertainty, actions may occasionally fail or result in unintended movement.

Environment Characteristics

- Partially observable
- Stochastic

Procedure

1. Introduce randomness in action outcomes using Python's **random** module.
2. Assign probabilities to successful and failed movements.
3. Execute multiple simulations of the same agent.
4. Compare different trajectories across runs.

Expected Learning Outcome

Students will analyze how **uncertainty and randomness** affect agent performance and decision reliability.

TASK 7: IMPLEMENTATION OF A SIMPLE REFLEX AGENT (RULE-BASED)

Objective

Implement a **simple reflex agent** that maps percepts directly to actions using condition-action rules.

Problem Description

A temperature monitoring system observes environmental temperature and activates a cooling mechanism when the temperature exceeds a threshold.

Environment Characteristics

- Fully observable
- Deterministic

Procedure

1. Simulate temperature values as input percepts.
2. Define condition-action rules:
    - If temperature > threshold → Activate cooling
    - Else → Maintain current state
3. Implement the simple reflex agent using if-else logic.
4. Test the agent against multiple temperature readings.

Expected Learning Outcome

Students will differentiate **simple reflex agents** from more complex agent models and understand their limitations.

TASK 8: COMPARATIVE ANALYSIS OF AGENT–ENVIRONMENT INTERACTIONS

Objective

Conduct a comparative evaluation of agent performance across different agent architectures and environment models.

Procedure

1. Execute:
    - Simple reflex agent in deterministic environment
    - Model-based agent in stochastic environment
2. Record:
    - Number of actions taken
    - Success or failure
3. Compare results and provide observations.

Expected Learning Outcome

Students will develop an analytical understanding of **agent suitability** with respect to environment complexity.

Submission Guidelines:

1. Download the .ipynb file from Jupyter Notebook and push it to your GitHub repository named **yourRollNumber_Notebooks** (use the same repository created for the previous task).
2. Create a **Word report** documenting **all the steps** you performed while creating the **n8n workflow**.
    - Include **screenshots for each step** showing both **configuration** and **successful execution**.
    - Place screenshots clearly under their respective steps.
3. Paste the **GitHub repository link at the top of the document**, convert the document to **PDF**, and upload it to **GCR**.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the total marks.**